

Localidade temporal e espacial de cache

Implemente duas versões da multiplicação de matriz por vetor (MxV) em C: uma com acesso à matriz por linhas (laço interno variando linha). Meça o tempo de execução de cada versão com uma função apropriada e execute testes com diferentes tamanhos de matriz. Identifique a partir de que tamanho os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações com o uso da memória cache e o padrão de acesso à memória.

- O gargalo de von Neumman e a memória cache
 - Localidade temporal e espacial
 - Row-major vs column-major

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

double get_time_diff(struct timespec start, struct timespec end)
{
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec)
/ 1e9;
};

int main () {
    int sizes[] = {256, 512, 1024, 2048, 4096, 8192, 16384, 32768};
    int num_sizes = sizeof(sizes)/sizeof(sizes[0]);

    for (int s=0; s < num_sizes; s++) {
        int n = sizes[s];

        int (*A)[n] = malloc(sizeof(int)[n][n]);
        int *X = malloc(n*sizeof(int));
        int *Y_row = calloc(n, sizeof(int));
        int *Y_col = calloc(n, sizeof(int));

        if (!A || !X || !Y_row || !Y_col){
            printf("Erro na alocação: %d\n", n);
            return 1;
        }

        for (int i = 0; i < n; i++){
            X[i] = rand() % 100;
            for (int j = 0; j < n; j++){
```

```

        A[i][j] = rand() % 100;
    }
}

struct timespec start, end;
double time_row, time_col;

clock_gettime (CLOCK_MONOTONIC, &start);
for (int i=0; i<n; i++){
    for (int j=0; j<n; j++){
        Y_row[i] += A[i][j]* X[j];
    }
}
clock_gettime (CLOCK_MONOTONIC, &end);
time_row = get_time_diff(start, end);

clock_gettime (CLOCK_MONOTONIC, &start);
for (int j=0; j<n; j++){
    for (int i=0; i<n; i++){
        Y_col[i] += A[i][j]* X[j];
    }
}
clock_gettime (CLOCK_MONOTONIC, &end);
time_col = get_time_diff(start, end);

printf("%-10d | %-20.6f | %-20.6f | %-10.2f\n", n, time_row,
time_col, time_col / time_row);

    free(A);
    free(X);
    free(Y_row);
    free(Y_col);
}
return 0;
}

```

Saída:

256	0.000117	0.000126	1.08
512	0.000488	0.000570	1.17
1024	0.001874	0.003200	1.71
2048	0.007557	0.020366	2.70
4096	0.029901	0.077064	2.58
8192	0.121379	0.435254	3.59
16384	0.522422	2.313993	4.43
32768	2.447682	12.799574	5.23

Respondendo a pergunta da atividade:

Os tempos passam a divergir de forma visível a partir do tamanho $N = 1024$ (onde o Column-major já é 1.71x mais lento). No entanto, o salto mais significativo e agressivo ocorre em $N = 2048$, com a diferença subindo bruscamente para 2.70x. A degradação continua escalando até atingir uma ineficiência massiva de 5.23x em $N = 32768$.

Para $N \leq 512$:

A matriz inteira ocupa até a aproximadamente 1 MB, cabendo completamente nas memórias cache L2 e L3. Como as consultas não chegam à RAM, os tempos de ambos os algoritmos são quase idênticos.

Para $N > 512$:

A matriz ultrapassa 4 MB e não cabe mais na cache. O método Col-major começa a expor o gargalo de von Neumann devido à quebra da localidade espacial. O desempenho degrada severamente até atingir uma execução 5.23x mais lenta em $N = 32768$.

Obs:

Em 65536 o PC precisou reiniciar pois uma matriz desse tamanho exige mais de 17 GB de RAM apenas para alocação, esgotando a memória física total do sistema e causando uma falha crítica por falta de recursos.

Gargalo de Von Neumman:

Devido a uma limitação física na arquitetura dos computadores, que separa a Unidade Central de Processamento (CPU) da Memória Principal (RAM). Como a CPU processa dados em uma velocidade muito superior à capacidade da RAM de fornecê-los, os dados precisam transitar por um barramento (bus) de comunicação que é comparativamente lento. A CPU passa grande parte do tempo ociosa, esperando a chegada dos dados. Para mitigar esse gargalo, introduziu-se a Memória Cache, uma memória menor, porém ultrarrápida, localizada muito próxima ou dentro da CPU, que armazena cópias dos dados mais acessados da RAM.

Row-major (Linha principal): A matriz é armazenada linha por linha. O elemento $A[0][0]$ fica fisicamente ao lado do $A[0][1]$. É o padrão nativo da linguagem C.

Column-major (Coluna principal): A matriz é armazenada coluna por coluna. O elemento $A[0][0]$ fica fisicamente ao lado do $A[1][0]$. É o padrão de linguagens como Fortran.

Em C:

No Row-Major o programa lê $A[0][0]$, sofre um atraso inicial para buscar na RAM, mas os próximos 15 elementos ($A[0][1]$, $A[0][2]$, etc.) já foram trazidos para a Cache no mesmo bloco. Já no Col-major programa lê $A[0][0]$, traz um bloco de 64 bytes para a Cache, mas logo em seguida exige o $A[1][0]$. Como a matriz está armazenada por linhas, o $A[1][0]$ está fisicamente distante na RAM. A CPU ignora os dados que trouxe para a Cache, é forçada a ir na RAM novamente e descarta o bloco anterior. Isso gera uma alta taxa de falhas de cache (Cache Misses), derrubando o desempenho.

Pq Row Major foi mais rápido:

O compilador C resolve isso gravando a matriz em formato Row-major: ele pega a primeira linha inteira e grava de forma reta e contígua, depois cola a segunda linha logo à frente, e assim por diante. Quando a CPU solicita um único dado da RAM, o controlador de memória traz um pacote inteiro contendo o dado solicitado e vários de seus vizinhos físicos imediatos. o usar o algoritmo Row-major (onde o laço interno avança na mesma linha: $A[0][0]$, $A[0][1]$, $A[0][2]$), o programa pede o primeiro dado, sofre um leve atraso indo à RAM, mas os próximos dados já vieram de brinde no mesmo pacote e estão na memória Cache que é muito rápida.

Escolha de Valores:

O uso de um array explícito `sizes[] = {256, 512, 1024, 2048...}` dobrando os valores foi adotado porque o hardware e as memórias cache são estruturados em potências de 2 (binário). Ao dobrar o N, quadruplicamos a área da matriz ($O(N^2)$), o que permite ultrapassar os limites de tamanho das caches. Isso mostra o exato momento em que os dados param de caber na cache rápida e vão para a lenta.

Função clock_time_diff:

A função clock_gettime retorna o tempo em uma estrutura dupla: tv_sec armazena os segundos inteiros e tv_nsec armazena os nanossegundos. A equação unifica essas duas grandezas em um único número decimal (tipo double):

1. $(\text{end.tv_sec} - \text{start.tv_sec})$: Calcula a diferença em segundos inteiros.
2. $(\text{end.tv_nsec} - \text{start.tv_nsec}) / 1\text{e}9$: Calcula a diferença dos nanossegundos e a converte para segundos. O valor 1e9 é a notação científica para 10^9 (1 bilhão), pois 1 segundo contém 1 bilhão de nanossegundos.
3. A soma das duas partes resulta no tempo total exato em segundos (ex: $2 + 0.5 = 2.5$ segundos).